

```

import pandas as pd
import numpy as np
import itertools
# plotting data - matplotlib
from matplotlib import pyplot as plt
# time series - statsmodels
# Seasonality decomposition
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.seasonal import seasonal_decompose
# holt winters
# single exponential smoothing
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
# double and triple exponential smoothing
from statsmodels.tsa.holtwinters import ExponentialSmoothing
# metrics
from sklearn.metrics import mean_absolute_error

```

```

airline = pd.read_csv('airline-passengers_data.csv', index_col='Month', parse_dates=True)
airline.rename(columns = {'#Passengers':'Passengers'}, inplace = True)
# finding shape of the dataframe
print(airline.shape)
# having a look at the data
print(airline.head())

```

```

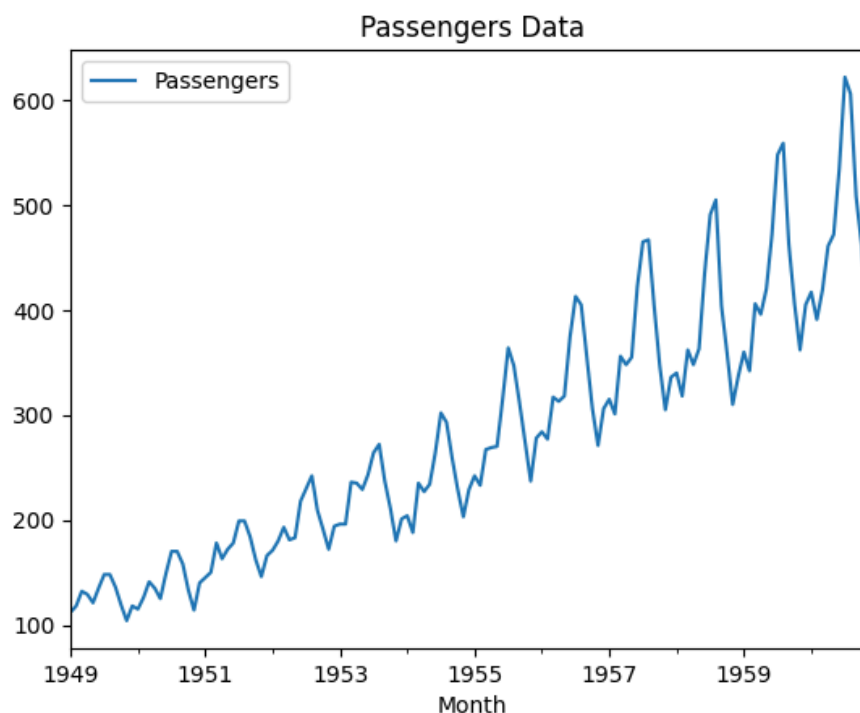
(144, 1)
      Passengers
Month
1949-01-01      112
1949-02-01      118
1949-03-01      132
1949-04-01      129
1949-05-01      121

```

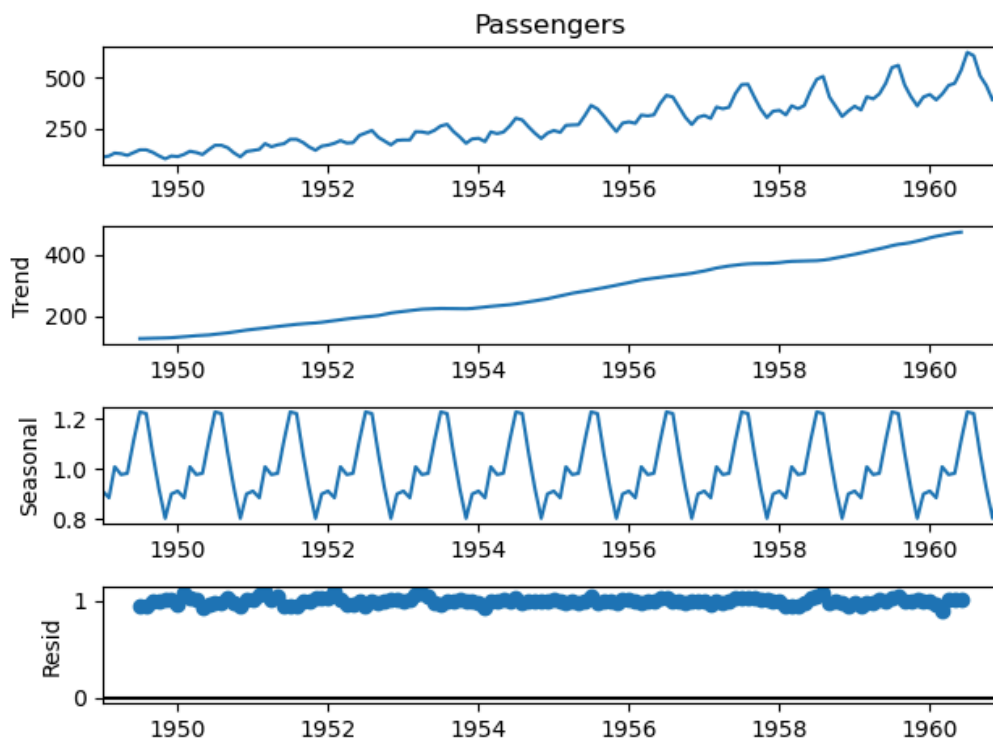
Start coding or [generate](#) with AI.

```
airline[['Passengers']].plot(title='Passengers Data')
```

<Axes: title={'center': 'Passengers Data'}, xlabel='Month'>



```
decompose_result = seasonal_decompose(airline[ 'Passengers'],model='multiplicative')
decompose_result.plot()
plt.show()
```



```
alphas = [0.1, 0.2, 0.9]

# Plot the original data
plt.figure(figsize=(10, 6))
plt.plot(airline, label='Original Data')

# Plot smoothed curves for different alphas
for alpha in alphas:
    model = SimpleExpSmoothing(airline).fit(smoothing_level=alpha)
    smoothed_data = model.fittedvalues
    plt.plot(smoothed_data, label=r'$\alpha$ = {}'.format(alpha))

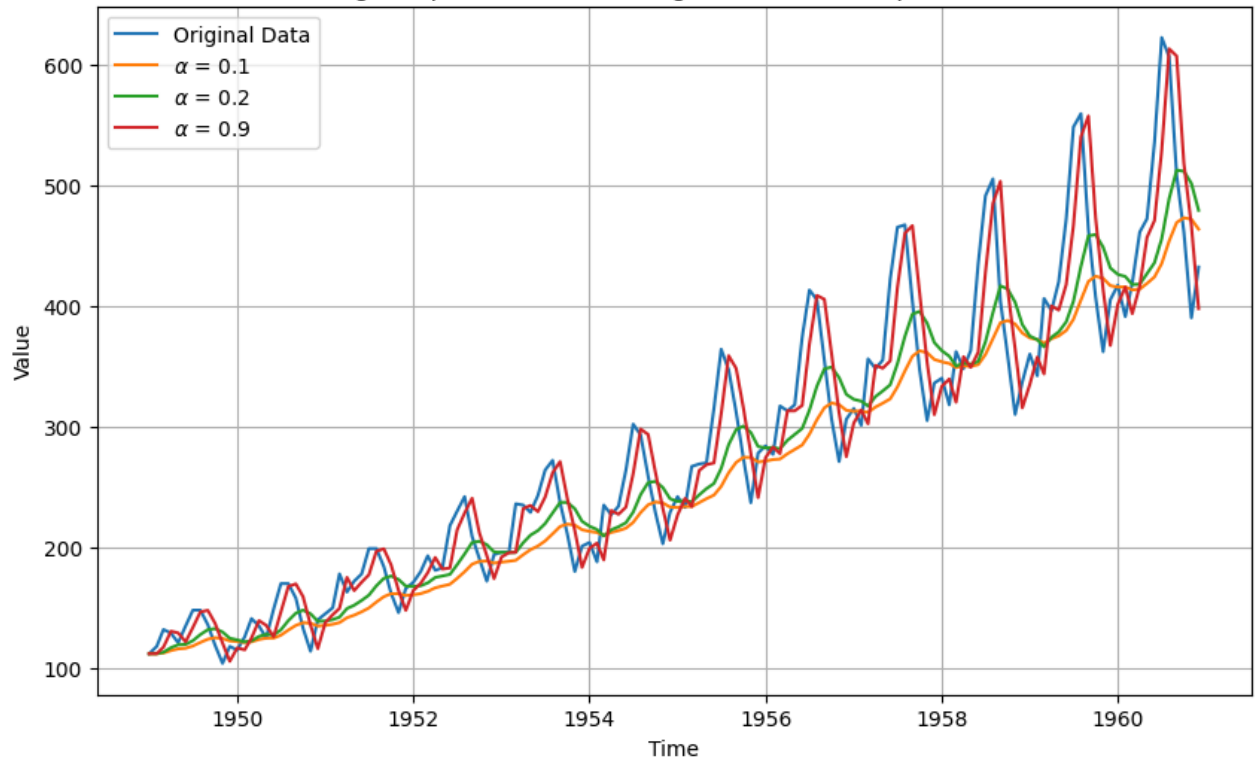
plt.title('Single Exponential Smoothing with Different Alpha Values')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.show()
```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/pandas/util/_decorators.py:213: EstimationWarning: Model has no
return func(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/pandas/util/_decorators.py:213: EstimationWarning: Model has no
return func(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/pandas/util/_decorators.py:213: EstimationWarning: Model has no
return func(*args, **kwargs)

```

Single Exponential Smoothing with Different Alpha Values



```

# SES paramters
# initialization_method - to initialize paramters
# None

# 'estimated'

# 'heuristic'

# 'legacy-heuristic'

# 'known'

# fit method parameters

# smoothing_levelfloat, optional
# The smoothing_level value of the simple exponential smoothing, if the value is set then this value

# optimizedbool, optional
# Estimate model parameters by maximizing the log-likelihood.

# use_brutebool, optional
# Search for good starting values using a brute force (grid) optimizer. If False, a naive set of star

# Define beta values
betas = [0.1, 0.2, 0.3, 0.9]

```

```

# Plot the original data
plt.figure(figsize=(10, 6))
plt.plot(airline, label='Original Data')

# Define colors for each beta value
colors = ['blue', 'green', 'red', 'orange']

# Plot smoothed curves for different betas with different colors
for beta, color in zip(betas, colors):
    model = ExponentialSmoothing(airline, trend='add').fit(smoothing_level=beta)
    smoothed_data = model.fittedvalues
    plt.plot(smoothed_data, label=r'$\beta$ = {}'.format(beta), color=color)

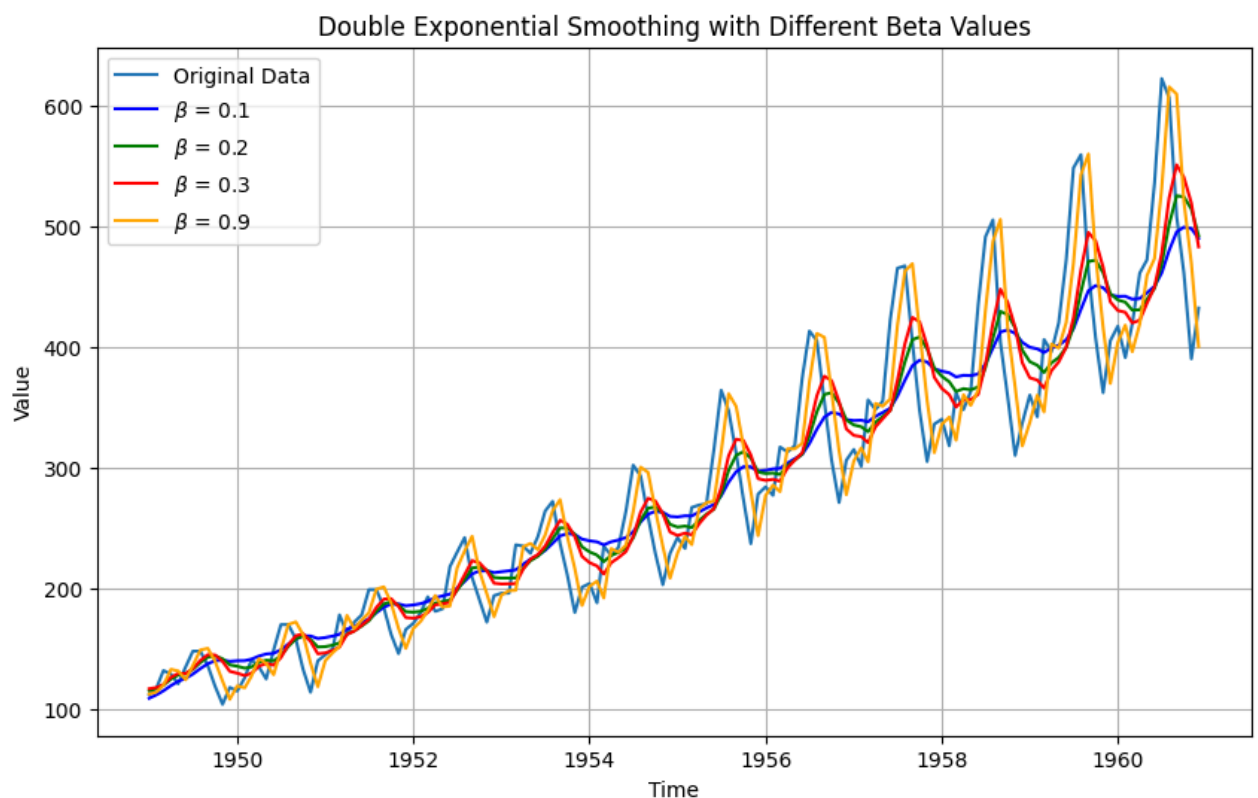
plt.title('Double Exponential Smoothing with Different Beta Values')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency specified
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency specified
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency specified
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency specified
self._init_dates(dates, freq)

```



```

# Define beta values
betas = [0.1, 0.2, 0.3, 0.9]

# Plot the original data
plt.figure(figsize=(10, 6))
plt.plot(airline, label='Original Data')

# Define colors for each beta value

```

```

colors = ['blue', 'green', 'red', 'orange']

# Plot smoothed curves for different betas with different colors
for beta, color in zip(betas, colors):
    model = ExponentialSmoothing(airline, trend='mul').fit(smoothing_level=beta)
    smoothed_data = model.fittedvalues
    plt.plot(smoothed_data, label=r'$\beta$ = {}'.format(beta), color=color)

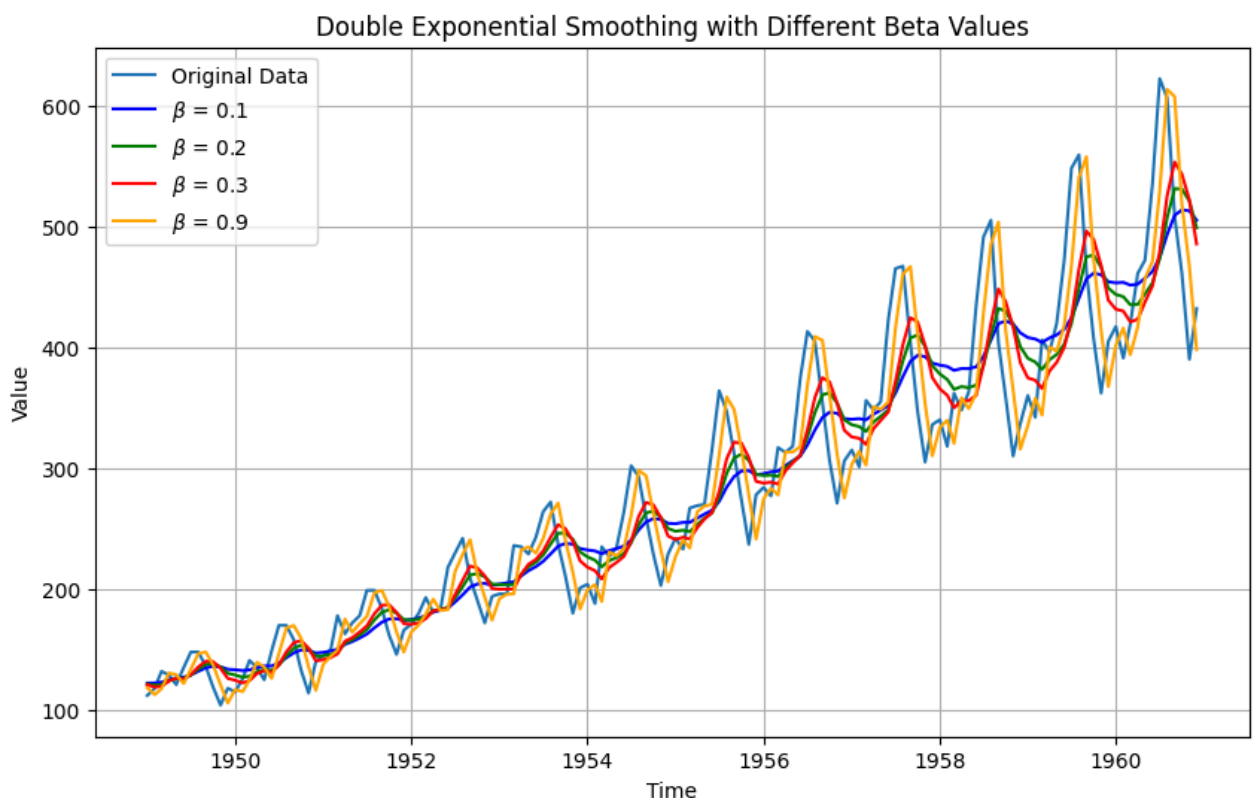
plt.title('Double Exponential Smoothing with Different Beta Values')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency
self._init_dates(dates, freq)

```



```
# Triple exponential hyperparameter tuning
```

```
airline.index.freq = 'MS'
```

```
train=airline[:132]
test=airline[132:]
```

```

def tes_optimizer(train, test, abg, trend_mode='add', seasonal_mode = 'mul', seasonal_period=12,step=

    best_alpha, best_beta, best_gamma, best_mae = None, None, None, float("inf")
    for comb in abg:
        tes_model = ExponentialSmoothing(train, trend=trend_mode, seasonal=seasonal_mode, seasonal_

```

```

fit(smoothing_level=comb[0], smoothing_trend=comb[1], smoothing_seasonal=comb[2])
y_pred = tes_model.forecast(step)
mae = mean_absolute_error(test, y_pred)
if mae < best_mae:
    best_alpha, best_beta, best_gamma, best_mae = comb[0], comb[1], comb[2], mae
print([round(comb[0], 2), round(comb[1], 2), round(comb[2], 2), round(mae, 2)])
print("best_alpha:", round(best_alpha, 2), "best_beta:", round(best_beta, 2), "best_gamma:", round(
    "best_mae:", round(best_mae, 4))
return best_alpha, best_beta, best_gamma, best_mae

```

Start coding or [generate](#) with AI.

```

alphas = betas = gammas = np.arange(0.10, 1, 0.20)
abg = list(itertools.product(alphas, betas, gammas))

```

```
best_alpha, best_beta, best_gamma, best_mae = tes_optimizer(train, test, abg)
```

```

[np.float64(0.1), np.float64(0.1), np.float64(0.1), 19.35]
[np.float64(0.1), np.float64(0.1), np.float64(0.3), 14.68]
[np.float64(0.1), np.float64(0.1), np.float64(0.5), 12.26]
[np.float64(0.1), np.float64(0.1), np.float64(0.7), 12.23]
[np.float64(0.1), np.float64(0.1), np.float64(0.9), 11.52]
[np.float64(0.1), np.float64(0.3), np.float64(0.1), 17.97]
[np.float64(0.1), np.float64(0.3), np.float64(0.3), 20.37]
[np.float64(0.1), np.float64(0.3), np.float64(0.5), 21.78]
[np.float64(0.1), np.float64(0.3), np.float64(0.7), 22.36]
[np.float64(0.1), np.float64(0.3), np.float64(0.9), 23.69]
[np.float64(0.1), np.float64(0.5), np.float64(0.1), 40.54]
[np.float64(0.1), np.float64(0.5), np.float64(0.3), 36.72]
[np.float64(0.1), np.float64(0.5), np.float64(0.5), 29.59]
[np.float64(0.1), np.float64(0.5), np.float64(0.7), 21.14]
[np.float64(0.1), np.float64(0.5), np.float64(0.9), 16.78]
[np.float64(0.1), np.float64(0.7), np.float64(0.1), 41.82]
[np.float64(0.1), np.float64(0.7), np.float64(0.3), 28.05]
[np.float64(0.1), np.float64(0.7), np.float64(0.5), 11.29]
[np.float64(0.1), np.float64(0.7), np.float64(0.7), 35.84]
[np.float64(0.1), np.float64(0.7), np.float64(0.9), 17.91]
[np.float64(0.1), np.float64(0.9), np.float64(0.1), 32.68]
[np.float64(0.1), np.float64(0.9), np.float64(0.3), 10.74]
[np.float64(0.1), np.float64(0.9), np.float64(0.5), 27.94]
[np.float64(0.1), np.float64(0.9), np.float64(0.7), 17.95]
[np.float64(0.1), np.float64(0.9), np.float64(0.9), 15.72]
[np.float64(0.3), np.float64(0.1), np.float64(0.1), 12.54]
[np.float64(0.3), np.float64(0.1), np.float64(0.3), 16.39]
[np.float64(0.3), np.float64(0.1), np.float64(0.5), 16.77]
[np.float64(0.3), np.float64(0.1), np.float64(0.7), 15.78]
[np.float64(0.3), np.float64(0.1), np.float64(0.9), 18.22]
[np.float64(0.3), np.float64(0.3), np.float64(0.1), 30.05]
[np.float64(0.3), np.float64(0.3), np.float64(0.3), 41.18]
[np.float64(0.3), np.float64(0.3), np.float64(0.5), 32.78]
[np.float64(0.3), np.float64(0.3), np.float64(0.7), 32.62]
[np.float64(0.3), np.float64(0.3), np.float64(0.9), 37.18]
[np.float64(0.3), np.float64(0.5), np.float64(0.1), 33.32]
[np.float64(0.3), np.float64(0.5), np.float64(0.3), 50.62]
[np.float64(0.3), np.float64(0.5), np.float64(0.5), 46.25]
[np.float64(0.3), np.float64(0.5), np.float64(0.7), 47.49]
[np.float64(0.3), np.float64(0.5), np.float64(0.9), 51.25]
[np.float64(0.3), np.float64(0.7), np.float64(0.1), 19.98]
[np.float64(0.3), np.float64(0.7), np.float64(0.3), 38.75]
[np.float64(0.3), np.float64(0.7), np.float64(0.5), 51.21]
[np.float64(0.3), np.float64(0.7), np.float64(0.7), 54.87]
[np.float64(0.3), np.float64(0.7), np.float64(0.9), 61.37]
[np.float64(0.3), np.float64(0.9), np.float64(0.1), 9.83]
[np.float64(0.3), np.float64(0.9), np.float64(0.3), 26.73]
[np.float64(0.3), np.float64(0.9), np.float64(0.5), 50.74]
[np.float64(0.3), np.float64(0.9), np.float64(0.7), 46.23]
[np.float64(0.3), np.float64(0.9), np.float64(0.9), 3074.18]
[np.float64(0.5), np.float64(0.1), np.float64(0.1), 12.98]
[np.float64(0.5), np.float64(0.1), np.float64(0.3), 19.77]

```

```
[np.float64(0.5), np.float64(0.1), np.float64(0.5), 28.05]
[np.float64(0.5), np.float64(0.1), np.float64(0.7), 28.34]
[np.float64(0.5), np.float64(0.1), np.float64(0.9), 24.98]
[np.float64(0.5), np.float64(0.3), np.float64(0.1), 18.61]
[np.float64(0.5), np.float64(0.3), np.float64(0.3), 28.26]
[np.float64(0.5), np.float64(0.3), np.float64(0.5), 28.15]
```

```
best_alpha, best_beta, best_gamma
```

```
(np.float64(0.30000000000000004),
 np.float64(0.9000000000000001),
 np.float64(0.1))
```

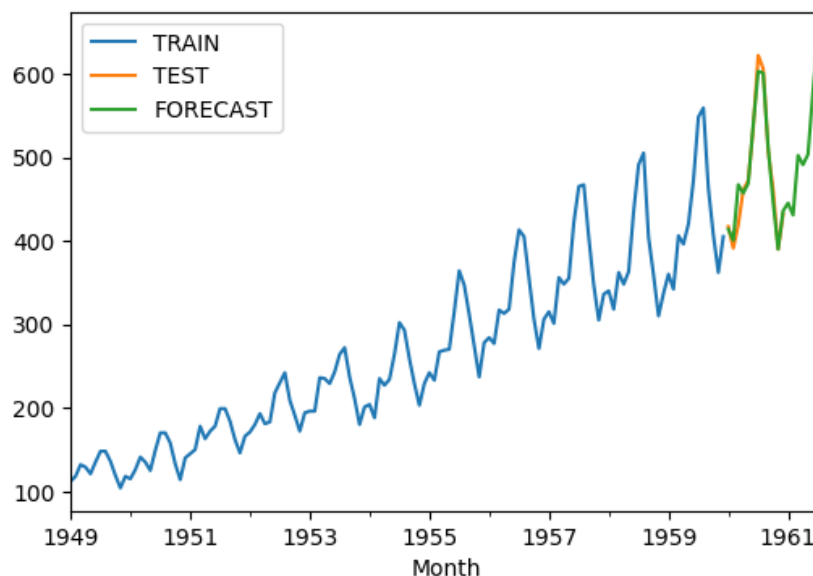
```
final_tes_model = ExponentialSmoothing(train, trend="add", seasonal="mul", seasonal_periods=12).\
    fit(smoothing_level=best_alpha, smoothing_trend=best_beta, smoothing_seasonal=best_gamma)
```

```
y_forecast = final_tes_model.forecast(20)
```

```
y_forecast.index
```

```
DatetimeIndex(['1960-01-01', '1960-02-01', '1960-03-01', '1960-04-01',
               '1960-05-01', '1960-06-01', '1960-07-01', '1960-08-01',
               '1960-09-01', '1960-10-01', '1960-11-01', '1960-12-01',
               '1961-01-01', '1961-02-01', '1961-03-01', '1961-04-01',
               '1961-05-01', '1961-06-01', '1961-07-01', '1961-08-01'],
              dtype='datetime64[ns]', freq='MS')
```

```
train['Passengers'].plot(legend=True,label='TRAIN')
test['Passengers'].plot(legend=True,label='TEST',figsize=(6,4))
y_forecast.plot(legend=True,label='FORECAST')
plt.show()
```



```
train.index
```

```
DatetimeIndex(['1949-01-01', '1949-02-01', '1949-03-01', '1949-04-01',
               '1949-05-01', '1949-06-01', '1949-07-01', '1949-08-01',
               '1949-09-01', '1949-10-01',
               ...,
               '1959-03-01', '1959-04-01', '1959-05-01', '1959-06-01',
               '1959-07-01', '1959-08-01', '1959-09-01', '1959-10-01',
               '1959-11-01', '1959-12-01'],
              dtype='datetime64[ns]', name='Month', length=132, freq='MS')
```

Start coding or [generate](#) with AI.

